

VRP Optimization Suite - Progress Report

🚧 Active Development - Modal Deployment Pending

This document outlines the architectural evolution and the features implemented during the transition from a local script-based prototype to a full-stack, serverless-ready web application.

🏗️ Architectural Evolution

Phase 1: Local Scripts

The original project consisted of sequential Python scripts (Geocoding, Matrix building, Routing) that read and wrote CSV/JSON files locally. It was single-user and ran synchronously.

Phase 2: Microservices (Docker)

Transitioned to a FastAPI backend using Redis and Celery to handle heavy VRP math asynchronously, preventing browser timeouts. Stored cache in PostGIS.

Phase 3: Serverless (Modal Target)

Refactored to be completely stateless, removing PostgreSQL, Redis, and Celery. Aiming to deploy the core FastAPI logic and heavy background tasks as serverless functions on Modal for infinite horizontal scaling.

🔧 Backend & Algorithmic Features Added

- **Advanced OOP Architecture:** Refactored the monolithic solver script into a highly modular object-oriented design using the **Strategy** and **Factory** design patterns.
- **BaseVRPSolver Interface:** Created an abstract base class to enforce a strict contract for all routing algorithms.
- **Algorithm Encapsulation:** Built dedicated `ORToolsSolver` and `ALNSSolver` classes to cleanly isolate the Google OR-Tools logic and the custom Adaptive Large Neighborhood Search heuristics.
- **Dynamic Strategy Selection:** The backend can now dynamically swap between OR-Tools, ALNS, or run them side-by-side in "Benchmark" mode based on frontend requests.

- **Stateless Geocoding:** Reworked the geocoding service to operate statelessly, ensuring compatibility with ephemeral serverless environments.
- **API Endpoint Streamlining:** Cleaned up the FastAPI router, consolidating the pipeline into a single `/upload-csv` trigger and a `/matrix-status` polling endpoint.

📱 Frontend Features Added

The frontend was completely rebuilt from scratch using React, Vite, Tailwind CSS, and Recharts, heavily styled with a premium, dark-mode glassmorphism aesthetic.

Landing & Configuration

- Drag-and-drop CSV upload zone.
- Dropdown for Algorithm Strategy selection.
- Inputs for Fleet Size and Vehicle Capacity.
- Advanced accordion for ALNS hyperparameters (Iterations, Segment Length, Destroy Bounds).

Loading State Machine

- Implemented a robust polling mechanism that pings the backend every 2 seconds.
- Sleek, animated loading screen that tracks backend progression without timing out.

Results Dashboard

- **KPI Ribbon:** Displays Success Rate, Fleet Utilization, Total Cost, and Unassigned Orders.
- **Interactive Chart:** Uses Recharts to plot a line graph comparing the cost reduction of OR-Tools vs ALNS over time.
- **Fleet Manifest:** Expandable cards showing stops, capacity, and duration for each vehicle.
- **Event Timeline:** A chronological log of how the simulation resolved orders.

📊 Current Status & Next Steps

The core logic, advanced algorithmic patterns, and the complete React frontend are **fully written and integrated**. The frontend successfully manages the flow from configuration to polling to results visualization.

Pending: The final deployment to Modal is currently facing environmental/configuration issues. Once the Modal deployment block is resolved, the application will be a live, highly scalable portfolio piece.